



Protocol Audit Report

Version 1.0

nuribnuu.vercel.app

May 21, 2026

Protocol Audit Report

Muhammad Nur Ibnu Hubab

May 21, 2025

Prepared by: Muhammad Nur Ibnu Hubab Lead Auditors:

- Muhammad Nur Ibnu Hubab

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
 - Issues found
- Findings
- High
 - [H-1] `PasswordStore::setPassword` has no access controls, meaning a non-owner could change the password
 - [H-2] Storing the password on-chain makes it visible to anyone, no longer private
- Informational
 - [I-1] The `PasswordStore::getPassword` natspec indicates a parameter that doesn't exist

Protocol Summary

A smart contract application for storing a password. Users should be able to store a password and then retrieve it later. Others should not be able to access the password.

Disclaimer

The Muhammad Nur Ibnu Hubab team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Commit Hash: [7d55682ddc4301a7b13ae9413095feffd9924566](#)

Scope

```
1 ./src/  
2 #- PasswordStore.sol
```

Roles

- Owner: The user who can set the password and read the password.
- Outsides: No one else should be able to set or read the password.

Issues found

Severiti	Number of issues found
High	2
Medium	0
Low	0
Info	1
Total	3

Findings

High

[H-1] PasswordStore::setPassword has no access controls, meaning a non-owner could change the password

Description: All data stored on-chain is visible to anyone, and can be read directly from the blockchain. The `PasswordStore::s_password` variable is intended to be a private intended to be only called by the owner of the contract.

We show one such method of reading any data off-chain below.

Impact: Anyone can read the private password, severely breaking the functionality of the protocol.

Proof of Concept: (Proof of Code)

The below test case shows how anyone can read the password directly from the blockchain.

1. Create a locally running chain

```
1 make anvil
```

2. Deploy the contract to the chain

```
1 make deploy
```

3. Run the storage tool We use 1 because that's the storage slot of `s_password` in the contract.

```
1 cast storage <ADDRESS_HERE> 1 --rpc-url http://127.0.0.1:8545
```

You'll get an output that looks like this: `0x6d7950617373776f726400000000000000000000000000000000000000000000`

```
1 cast parse-bytes32-string 0
   x6d7950617373776f72640000000000000000000000000000000000000000000014
2 myPassword
```

And get an output of:

```
1 myPassword
```

Recommended Mitigation: The overall architecture of the contract should be rethought. One possible approach is to encrypt the password off-chain and only store the encrypted version on-chain. This would require the user to remember a separate decryption key off-chain. Additionally, the `PasswordStore::getPassword` view function should be removed to prevent users from accidentally sending their decryption key as a transaction, which would expose it on-chain.

[H-2] Storing the password on-chain makes it visible to anyone, no longer private

Description: The `PasswordStore:setPassword` function is set to be an `external` function, however, the natspec of the function and overall purpose of the smart contract is that `This function allows only the owner to set a new password.`

```
1 function setPassword(string memory newPassword) external {
2 @> // @audit - there are no access controls
3     s_password = newPassword;
4     emit SetNetPassword();
5 }
```

Impact: Anyone can set the password of the contract, severely breaking the contract intended functionality.

Proof of Concept: Add the following to the `PasswordStore.t.sol` test file.

Code

```
1 function test_anyone_can_set_password(address randomAddress) public
   {
```

```
2      vm.assume(randomAddress != owner);
3      vm.prank(randomAddress);
4      string memory expectedPassword = "myNewPassword";
5      passwordStore.setPassword(expectedPassword);
6
7      vm.prank(owner);
8      string memory actualPassword = passwordStore.getPassword();
9      assertEq(actualPassword, expectedPassword);
10 }
```

Recommended Mitigation: Add an access control conditional to the `PasswordStore::setPassword` function.

```
1  if(msg.sender != owner) {
2      revert PasswordStore__NotOwner;
3  }
```

Informational

[I-1] The `PasswordStore::getPassword` natspec indicates a parameter that doesn't exist

Description:

```
1      /*
2      * @notice This allows only the owner to retrieve the password.
3      @> * @param newPassword The new password to set.
4      */
5      function getPassword() external view returns (string memory) {
```

The `PasswordStore::getPassword` function signature is `getPassword()` while the natspec says it should be `getPassword(string)`.

Impact: The natspec is incorrect.

Recommended Mitigation: Remove the incorrect natspec line.

```
1  - * @param newPassword The new password to set.
```